EXCELERATE MOBILE APP DEVELOPMENT WITH FLUTTER REMOTE INTERNSHIP

WEEK 3 DELIVERABLE REPORT

Excelerate Learning Hub Mobile Application

COVER PAGE

Submitted To

Excelerate Internship Program

Internship Track

Mobile App Development with Flutter Remote Internship

Module

Week 3: Dynamic Data Integration, State Management, Loading Indicators, Form Handling and Validation

Team Number

Team 4

Team Members

- Taiwo Jeje
- Obed Yakpa
- Benjamin Oben

Submission Date

October 2026

Dynamic Data Integration, State Management, Loading Indicators, Form Handling and Validation

EXECUTIVE SUMMARY

Week 3 focused on transforming the Excelerate Learning Hub application from a static user interface prototype into a more functional and interactive Flutter application. Building on the work completed in Week 2, we implemented dynamic data integration, state management, form validation, loading indicators, and error handling to improve the application's responsiveness and user experience.

A Program Model and Program Service were created to simulate real-world API integration using structured JSON data. The Program Listing and Program Details screens now display dynamic information instead of hardcoded content. A Feedback Form was also implemented to collect user input, supported by form validation, loading states, and submission confirmation messages.

To maintain consistency with the current Excelerate platform, the application was updated with the latest Excelerate branding, including the splash-style logo positioned consistently across all screens, branded headers, and the Excelerate color palette.

Overall, Week 3 provided practical experience in dynamic data handling, application state management, user interaction, and responsive Flutter development.

TABLE OF CONTENTS

Section	Page
Cover Page	i
Executive Summary	ii
Table of Contents	iii
1. Overview	1
2. Deliverable 1: API-Connected Functional Application	2
3. Deliverable 2: GitHub Repository Update	4
4. Deliverable 3: Changelog and Documentation	5
5. State Management	6

6. Testing and Validation	7
7. Evidence and Screenshots	8
8. Learning Outcomes Achieved	10
9. Reflection	11
10. Conclusion	12
References	13
Appendix A: Flutter Code Snippets	14

---

1. OVERVIEW

Week 3 marked an important transition from interface design to application functionality. The focus was on creating a mobile application capable of handling dynamic content, user interactions, validation, and state changes.

Key features implemented during this phase included:

- Dynamic JSON data integration
- Program Model and Program Service
- Program Listing and Program Details screens
- Flutter state management using `setState()`
- Feedback Form creation
- Form validation
- Loading indicators and skeleton screens
- Error handling with retry functionality
- Submission success confirmation
- Updated Excelerate branding

---

2. DELIVERABLE 1: API-CONNECTED FUNCTIONAL APPLICATION

A. Dynamic Program Listing and Program Details

The Program Listing and Program Details screens were upgraded from static hardcoded content to dynamic data-driven screens.

A Program Model was created to represent program information, while a Program Service was implemented to simulate an API call using JSON data.

Sample Program Data

JSON

```
[
  {
    "title": "Digital Strategy Associate Internship",
    "duration": "6 Weeks",
    "scholarship": "$1,000"
  },
  {
    "title": "Global Learning Innovation Associate",
    "duration": "8 Weeks",
    "scholarship": "$1,000"
  }
]
```

Key Insight

Separating application data from user interface elements improves application scalability and closely simulates real-world API integration.

B. Feedback Form

A Feedback Form was implemented to support user interaction and data collection.

Form Fields

- Full Name
- Email Address
- Feedback Message

User Flow

```
Input
  ↓
Validation
  ↓
Loading State
  ↓
Submission
  ↓
Success Confirmation
```

Key Insight

The form transformed the application from an informational platform into an interactive system capable of receiving and processing user input.

C. Form Validation

Validation rules were implemented to prevent incomplete or inaccurate submissions.

Validation Rules

- Name is required
- Email is required
- Email must contain "@"
- Feedback field cannot be empty

Key Insight

Validation improves usability by helping users identify and correct input errors before submission.

3. DELIVERABLE 2: GITHUB REPOSITORY UPDATE

The GitHub repository was updated with all Week 3 enhancements.

Repository Updates

- Program Model implementation
- Program Service implementation

- Dynamic Program Listing
- Dynamic Program Details
- Feedback Form
- Loading indicators
- Error handling and retry logic
- Updated README documentation

Sample Commit Messages

```
Added Program model

Created Program service

Integrated JSON data source

Implemented dynamic program listing

Added loading indicators

Created feedback form

Implemented validation logic

Added retry functionality

Updated README documentation
```

Key Insight

Professional version control practices improve collaboration, progress tracking, and project maintainability.

---

4. DELIVERABLE 3: CHANGELOG AND DOCUMENTATION

Dynamic Data Integration

The application now loads program information from a structured JSON source through a Program Service.

Dynamic Screens

- Program Listing Screen
- Program Details Screen

Key Insight

Dynamic data loading creates a more realistic and maintainable application architecture.

Feedback Form Workflow

```
User Input
      ↓
Validation
      ↓
Loading
      ↓
Submission
      ↓
Success Confirmation
```

Key Insight

The workflow demonstrates real-world user interaction patterns within a mobile application.

Loading and Error Handling

Loading State

Implemented using:

```
CircularProgressIndicator()
```

Error Handling

Implemented using:

```
try/catch
```

alongside retry functionality.

Retry Flow

```
Load Data
    ↓
Error Occurs
    ↓
Display Error
    ↓
Retry
    ↓
Reload Data
```

Key Insight

Providing feedback during system operations improves transparency and user confidence.

---

5. STATE MANAGEMENT

State refers to data that changes while an application is running, such as loading status, program data, validation messages, and form submission outcomes.

For this project, Flutter's built-in **`setState()`** method was used to manage application state.

States Managed

- Loading State
- Program Data State
- Validation State
- Submission State
- Success State

- Error State

Example

```
setState(() {
  isLoading = true;
});
```

Key Insight

Flutter provides several built-in state management tools, including `setState()`, `ValueNotifier`, `ChangeNotifier`, and `InheritedWidget`. For Week 3, `setState()` was sufficient because it efficiently handles localized state changes and automatically updates the user interface.

Reflection

The most important lesson learned was understanding how state changes drive dynamic content, loading indicators, validation messages, retry functionality, and submission confirmations.

Demo video Link:
https://drive.google.com/file/d/1viM4CJBPFawltrG5gAaykkPZr3wT7uZ3/view?usp=sharing

GitHub Repository Link: https://github.com/deji-oladiran/excelerate-learning-hub

---

6. TESTING AND VALIDATION

Test Case	Expected Result	Status
Load Program Listing	Dynamic data loads successfully	Passed
View Program Details	Selected information displays correctly	Passed
Empty Form Submission	Validation errors displayed	Passed
Invalid Email Entry	Validation message displayed	Passed
Valid Form Submission	Success confirmation displayed	Passed
Loading State	Loading indicator displayed	Passed

Error State	Error screen displayed	Passed
Retry Functionality	Data reloads successfully	Passed

Testing Summary

The application successfully handled:

- Dynamic JSON data loading
- State updates
- Form validation
- User Submissions
- Loading indicators
- Error handling
- Retry functionality

---

7. EVIDENCE AND SCREENSHOTS

Figure 1

Home Dashboard with Updated Excelerate Branding

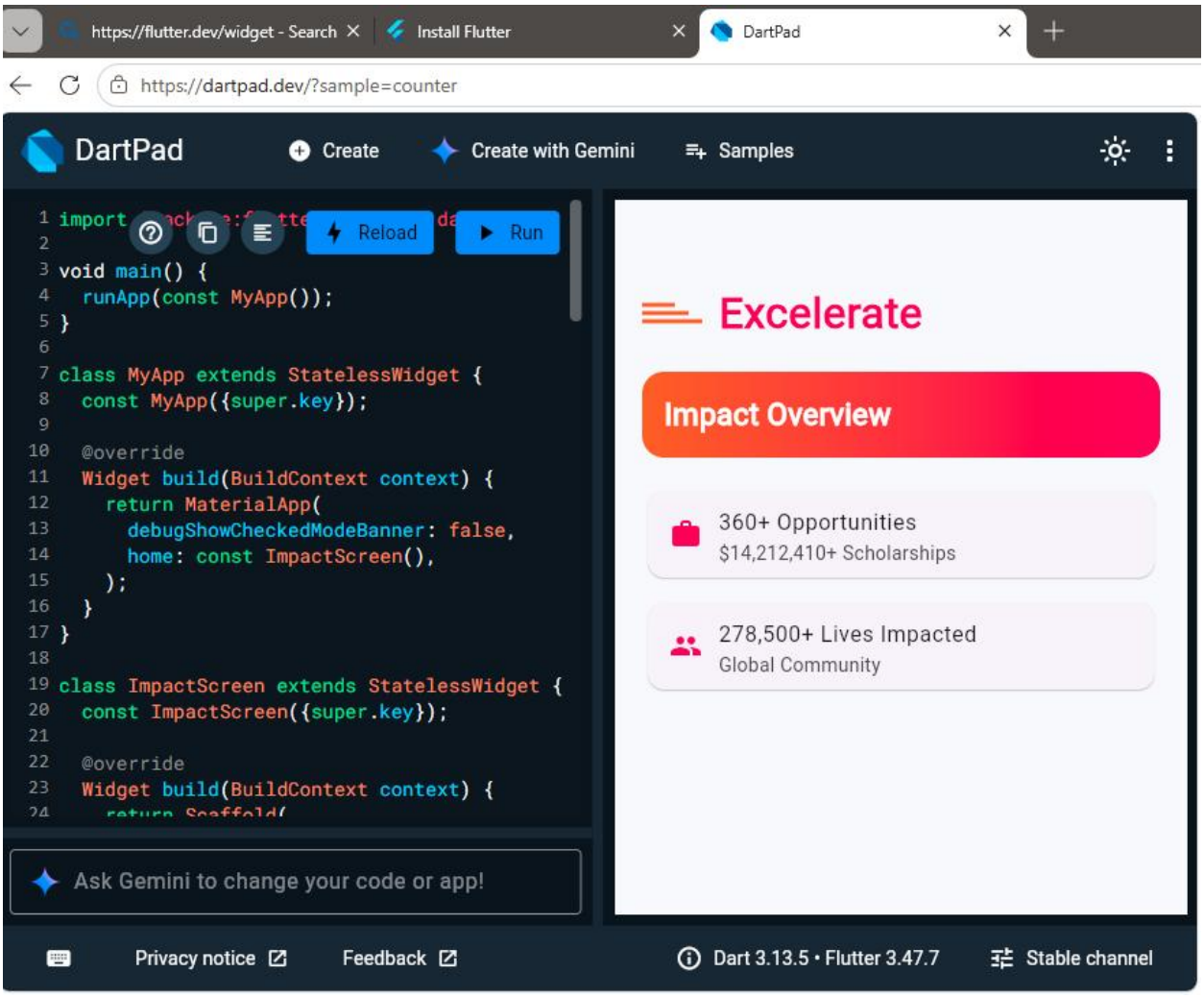


Figure 2

Achievements and Skills Dashboard

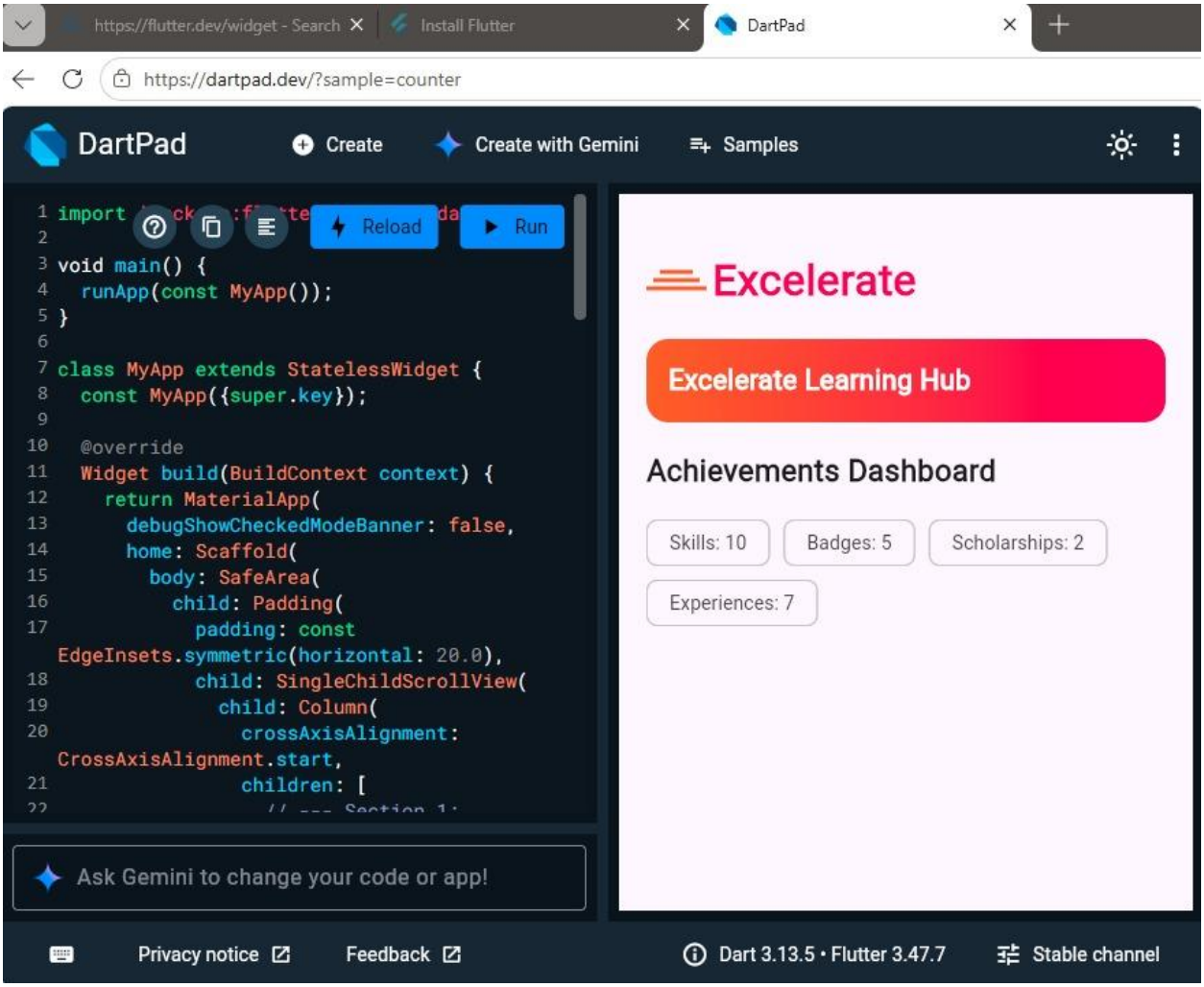


Figure 3

Dynamic Program Listing Screen

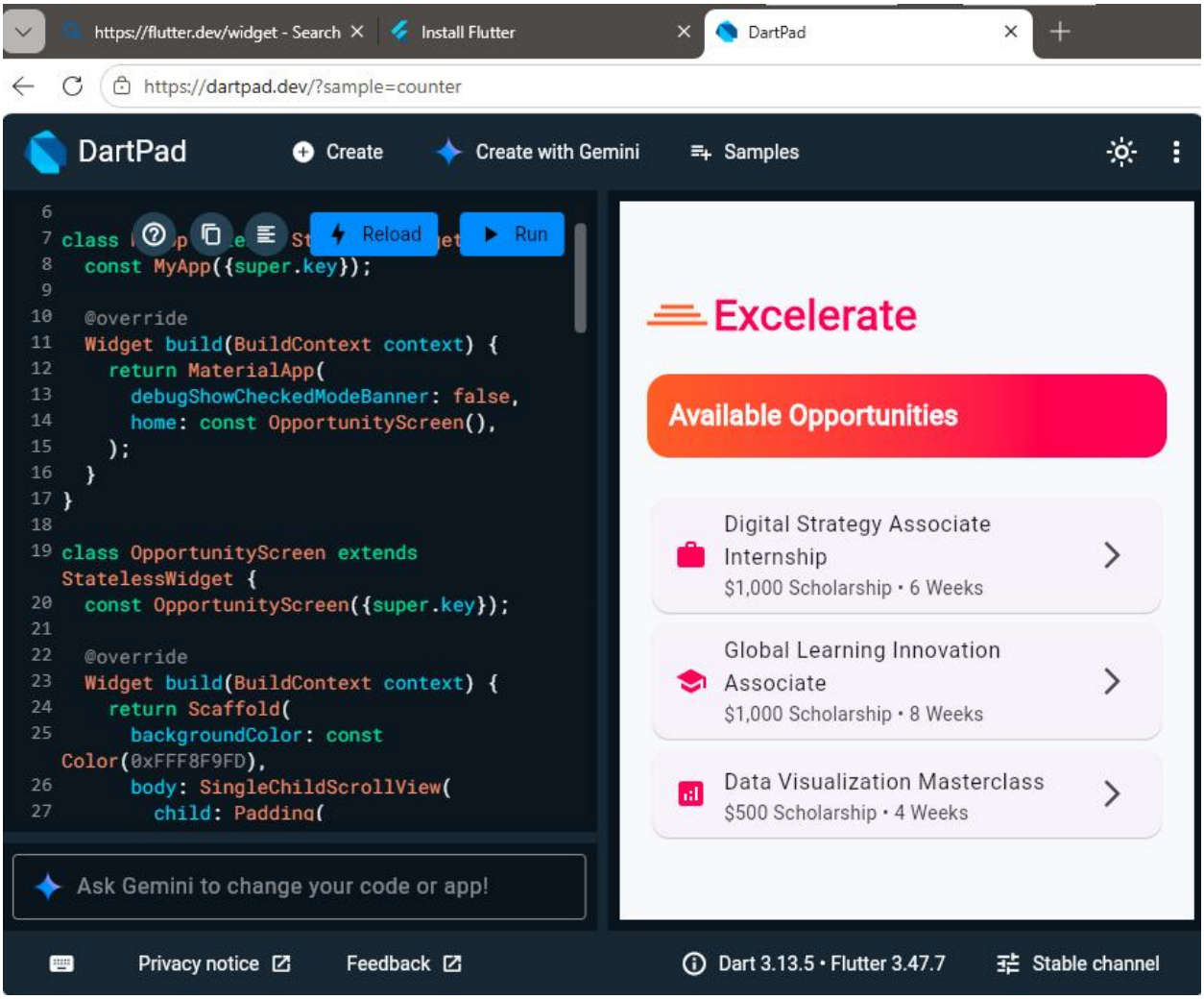


Figure 4

Feedback Form Interface

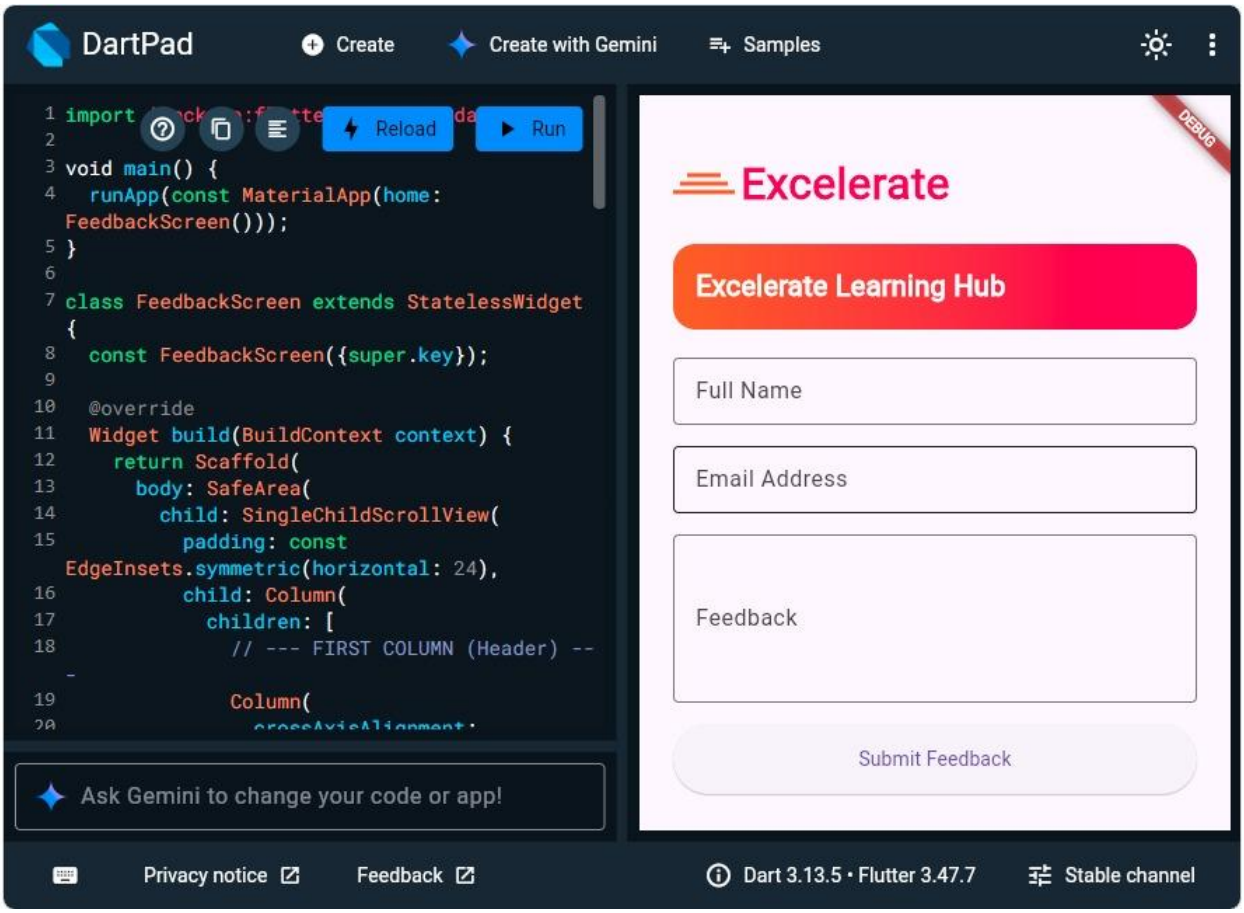


Figure 5

Loading Indicator State

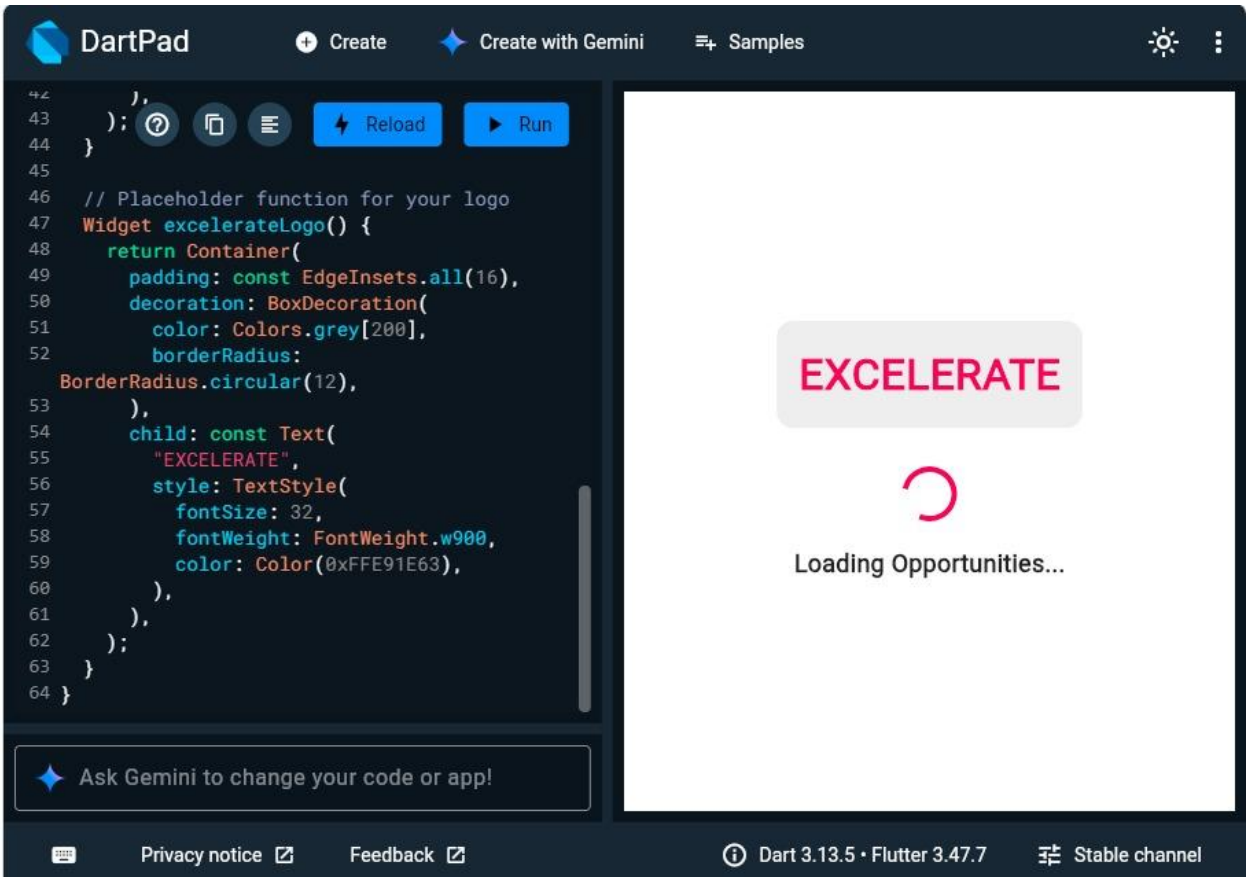


Figure 6

Skeleton Loading Screen

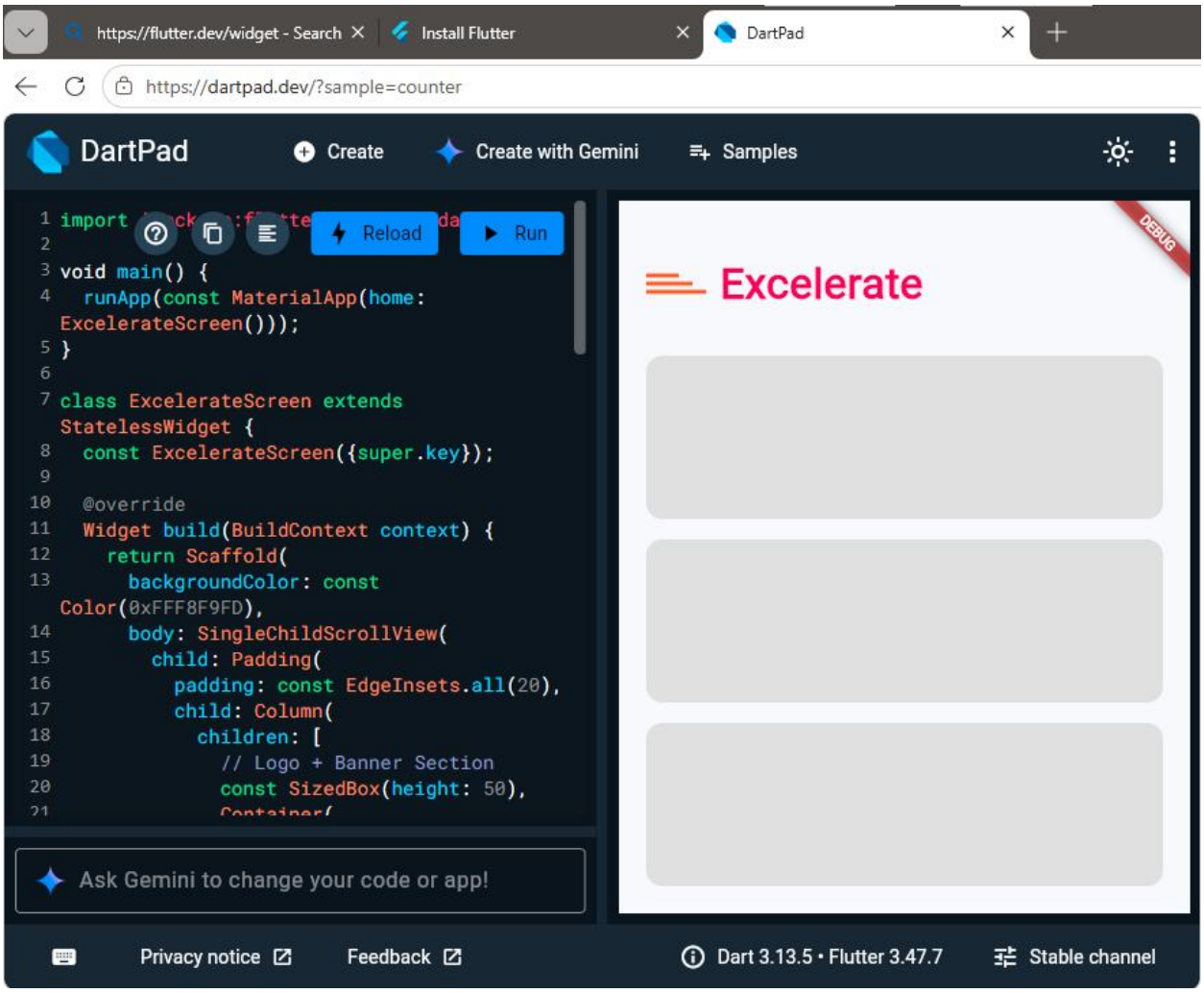


Figure 7

Dynamic Program Details Screen

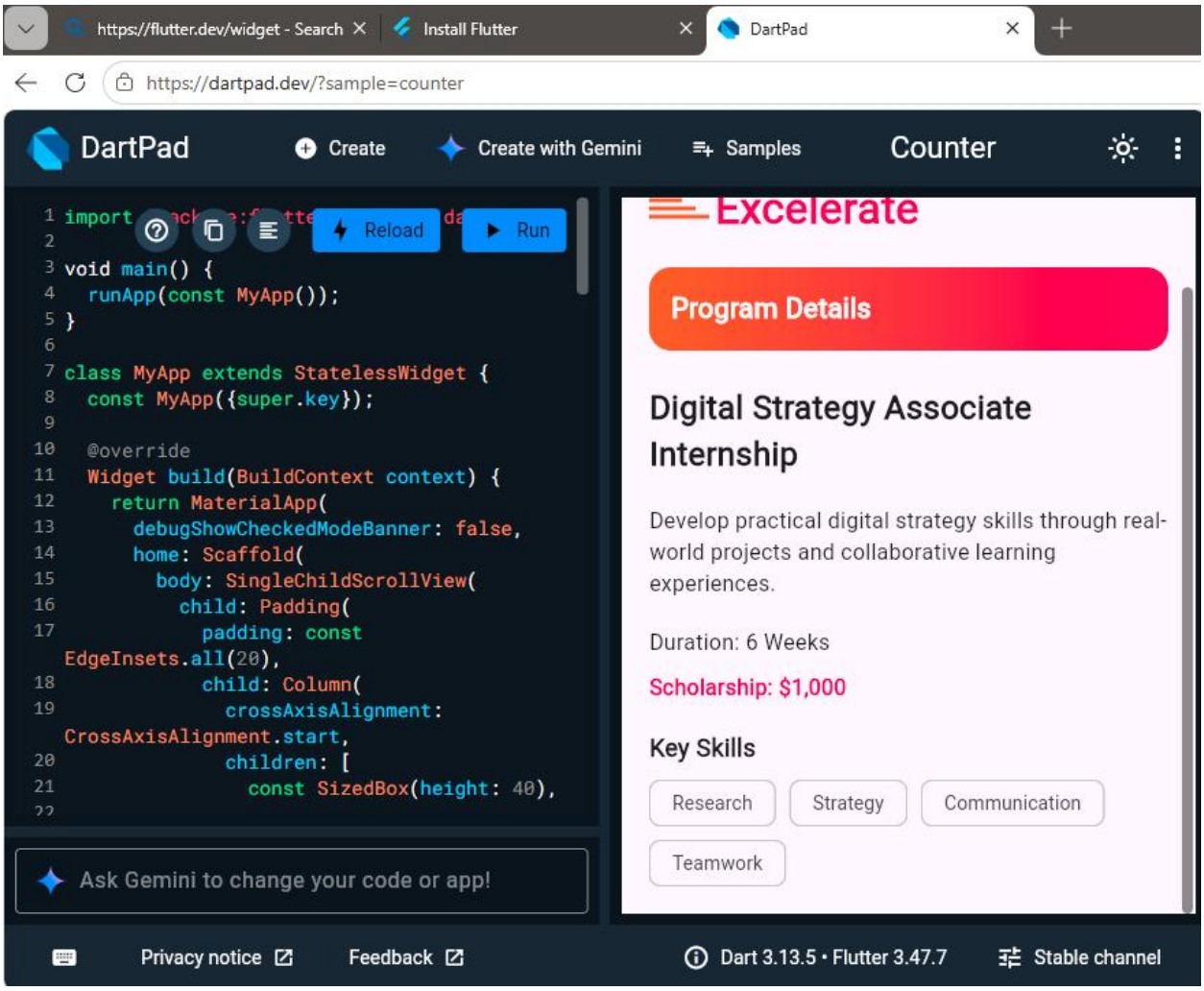


Figure 8

Successful Form Submission Confirmation

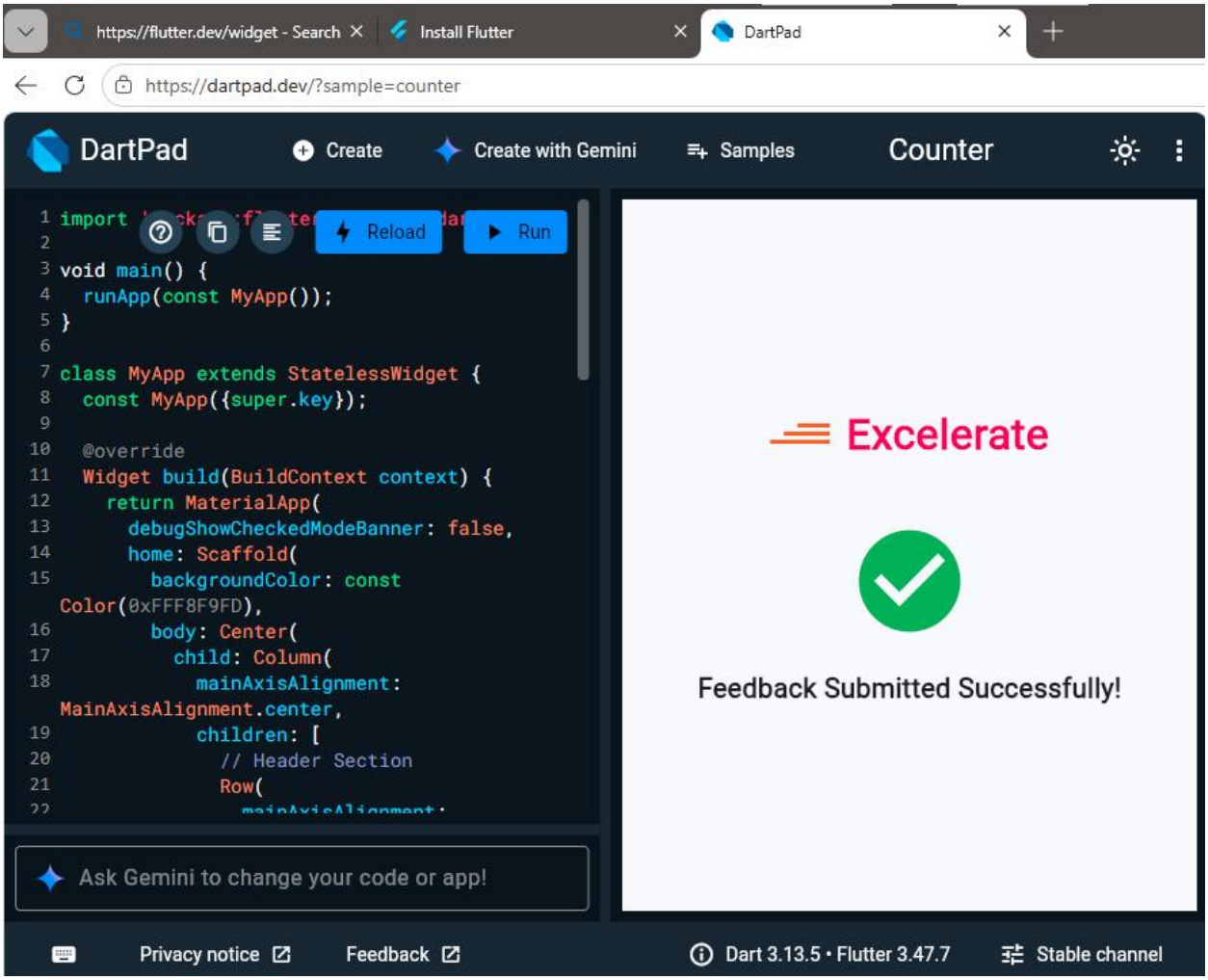


Figure 9

Form Validation State

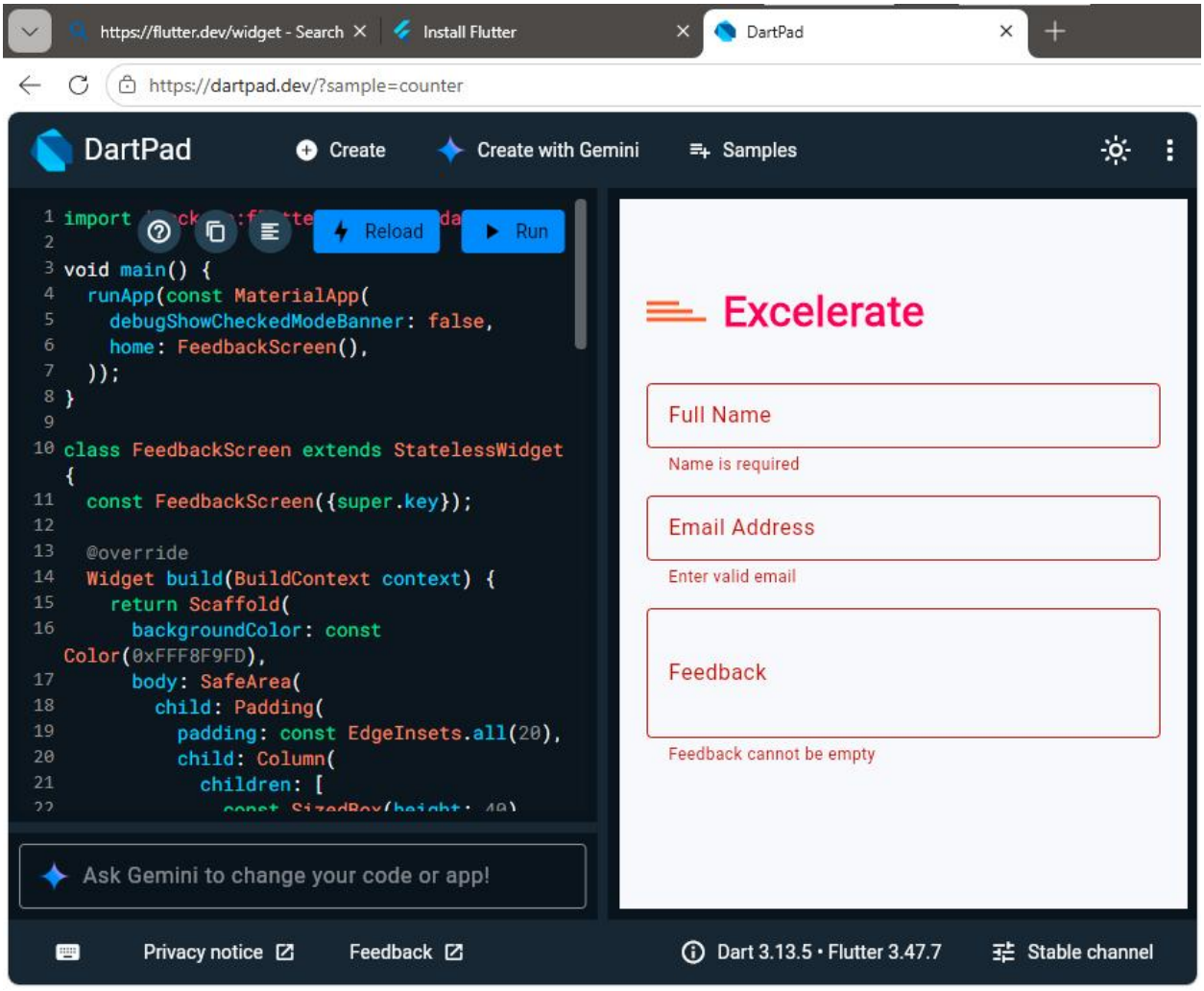
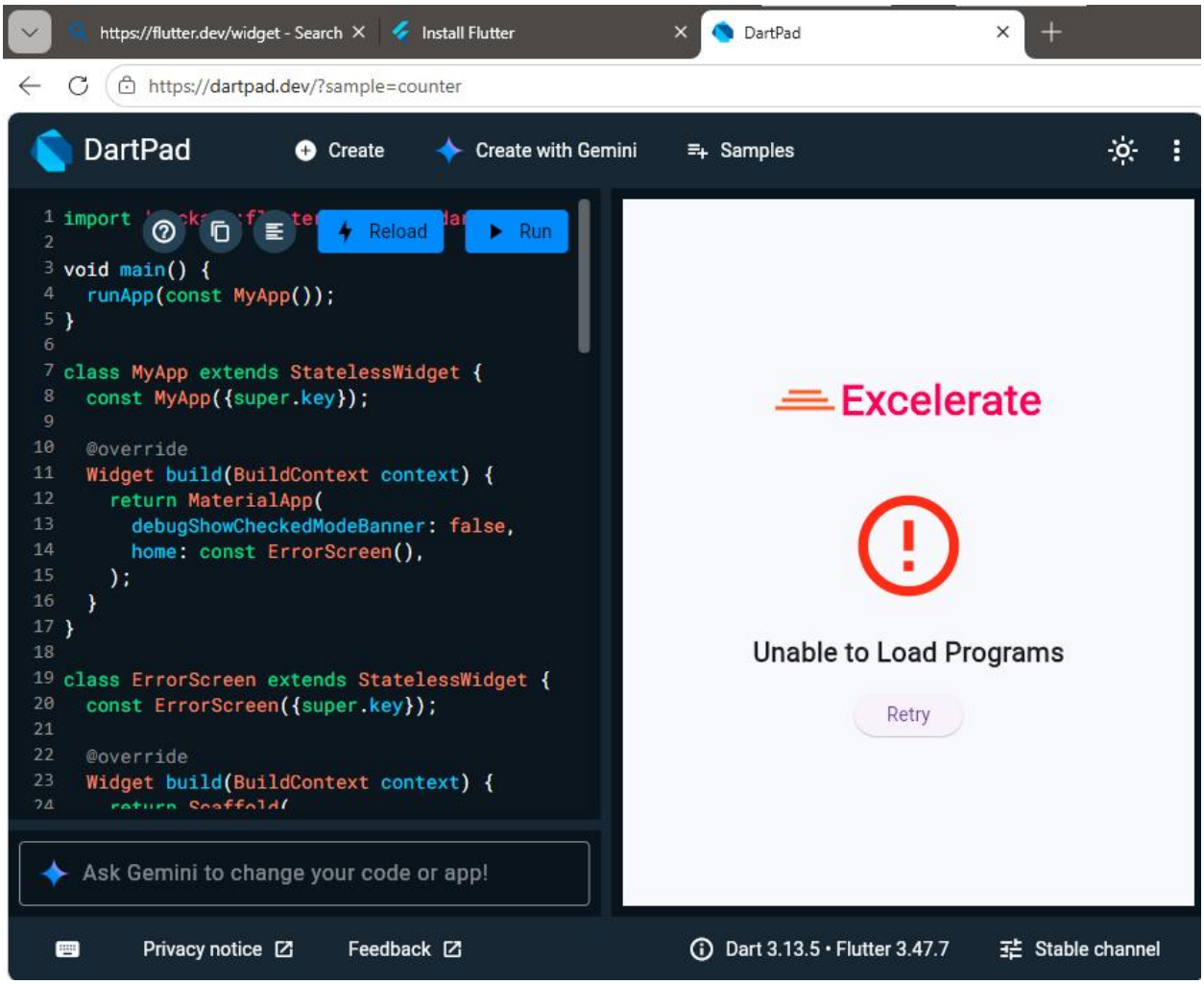


Figure 10

Error Handling and Retry Screen



Branding Update

The updated Excelerate splash-logo and wordmark were implemented consistently across all screens. The logo was positioned before page content and aligned with the visual identity of the current Excelerate platform.

---

8. LEARNING OUTCOMES ACHIEVED

By completing Week 3, Team 4 successfully:

. Connected Flutter screens to a JSON-based data source

. Created a Program Model and Program Service

- Displayed dynamic content instead of hardcoded information
- Implemented Flutter state management using `setState()`
- Added loading indicators and skeleton screens
- Implemented error handling and retry functionality
- Created a working feedback form
- Added validation and success confirmation messages
- Updated GitHub documentation professionally
- Built a more responsive and realistic Flutter application

---

9. REFLECTION

Week 3 represented a significant step forward in our Flutter development journey. While previous work focused primarily on user interface design, this phase introduced the functionality required to make the application behave like a real-world product.

Working with dynamic data, state management, validation, loading indicators, and error handling helped deepen our understanding of how modern mobile applications respond to user interactions and changing data. The experience also highlighted the importance of user feedback mechanisms and application reliability.

---

10. CONCLUSION

The Excelerate Learning Hub application successfully evolved from a static prototype into a functional Flutter application.

Through the implementation of a Program Model, Program Service, JSON data integration, state management, loading indicators, feedback forms, validation logic,

error handling, retry functionality, and updated Excelerate branding, the application now demonstrates the core concepts introduced in Week 3.

These enhancements provide a solid foundation for future development involving real APIs, backend integration, advanced state management, and production-ready mobile applications.

REFERENCES

Excelerate. (2026). *Week 3 action meeting notes: Mobile App Development with Flutter Remote Internship*.

Excelerate. (2026). *Week 3 SCORM module: Dynamic data integration, state management, loading indicators, and form handling*.

Flutter Developers. (2026). *CircularProgressIndicator class*. Flutter API Documentation. https://api.flutter.dev/flutter/material/CircularProgressIndicator-class.html

Flutter Developers. (2026). *Forms and validation cookbook*. Flutter Documentation. https://docs.flutter.dev/cookbook/forms/validation

Flutter Developers. (2026). *Manage state using setState*. Flutter Documentation. https://docs.flutter.dev

Flutter Developers. (2026). *Networking and asynchronous programming*. Flutter Documentation. https://docs.flutter.dev/cookbook/networking

Flutter Developers. (2026). *Text field changes and user input handling*. Flutter Documentation. https://docs.flutter.dev/cookbook/forms/text-field-changes

GitHub, Inc. (2026). *About commits and commit messages*. GitHub Docs. https://docs.github.com

---

APPENDIX A: FLUTTER CODE SNIPPETS

Include screenshots and corresponding Flutter code for:

- Figure 1: Home Dashboard
- Figure 2: Achievements Dashboard
- Figure 3: Dynamic Program Listing
- Figure 4: Feedback Form
- Figure 5: Loading State
- Figure 6: Skeleton Loading
- Figure 7: Program Details
- Figure 8: Validation State
- Figure 9: Submission Success
- Figure 10: Error Handling and Retry

Prepared By:
Team 4

Excelerate Learning Hub Mobile Application
Week 3 Deliverable Report